

Trabajo Práctico N° 1: Introducción a sockets TCP

Materia: Sistemas Distribuidos y Paralelos

Institución: Universidad Nacional de Villa Mercedes

Alumno: Marco Nicolas Barzola

1. Descripción de la Ejecución

Se implementó un servidor y un cliente TCP utilizando el módulo estándar `socket` de Python. El servidor funciona como un *echo server*, devolviendo exactamente los mismos bytes que recibe de los clientes. El cliente establece una conexión, permite enviar hasta cinco mensajes calculando el tiempo de ida y vuelta (RTT) mediante `time.perf_counter()`, y soporta el comando "SALIR" para realizar un cierre ordenado de la conexión. Todo el intercambio local se realizó sobre la dirección `127.0.0.1` en el puerto `65432`.

2. Salidas de Prueba

Prueba 1: Ejecución del servidor y recepción de cliente

```
Barzola@DESKTOP-1G76PT5 MINGW64 /f/INGENIERIA/Facu/Sistemas Distribuidos y Paralelos/Programacion-Socket
$ python servidor_echo.py
Servidor echo escuchando en 127.0.0.1:65432
Cliente conectado desde: ('127.0.0.1', 62917)
```

Prueba 2: Intercambio de mensajes y cálculo de RTT (Lado del Cliente)

```
Barzola@DESKTOP-1G76PT5 MINGW64 /f/INGENIERIA/Facu/Sistemas Distribuidos y Paralelos/Programacion-Socket
$ python cliente_echo.py
Mensaje 1 de 5 (escriba SALIR para terminar): Hola
Respuesta del servidor: Hola
RTT: 0.325 ms

Mensaje 2 de 5 (escriba SALIR para terminar):
```

Prueba 3: Comando SALIR y cierre ordenado (Lado del Cliente)

```
Barzola@DESKTOP-1G76PT5 MINGW64 /f/INGENIERIA/Facu/Sistemas Distribuidos y Paralelos/Programacion-Socket
$ python cliente_echo.py
Mensaje 1 de 5 (escriba SALIR para terminar): Hola
Respuesta del servidor: Hola
RTT: 0.325 ms

Mensaje 2 de 5 (escriba SALIR para terminar): Salir
Respuesta del servidor: Confirmación: Conexión finalizada.
RTT: 0.280 ms
```

3. Diagrama de Secuencia

El siguiente diagrama detalla el ciclo de vida de los sockets TCP implementados y el flujo de la comunicación:



